

Tổng Hợp Kiến Thức

1. Khái niệm cơ bản

- **Server Actions:** Hàm JavaScript bất đồng bộ chạy trên server, gọi được từ client/server components.
- Dùng để đơn giản hóa gọi API và cập nhật UI.
- Cú pháp: Hàm async với directive `"use server";` ở đầu file hoặc hàm.
- Tham số mặc định từ `<form>` : `FormData` object.

2. Tạo Server Action

- **Cú pháp cơ bản:**

```
"use server";

export async function myAction(formData) {
  const data = formData.get("fieldName");
  // Xử lý logic server
  return { success: true, message: "Thành công" };
}
```

- **Gắn vào form:**

```
import { myAction } from './actions';

<form action={myAction}>
  <input type="text" name="fieldName" required />
  <button type="submit">Submit</button>
</form>
```

3. Quản lý trạng thái với `useActionState`

- **Sử dụng:**

```
"use client";
import { useActionState } from "react";
import { myAction } from "./actions";

function MyComponent() {
  const [state, formAction] = useActionState(myAction, { message: null, success: null });

  return (
    <form action={formAction}>
      <input type="text" name="fieldName" />
      {state.message && <p>{state.message}</p>}
      <button type="submit">Submit</button>
    </form>
  );
}
```

- **Xử lý kết quả:**

```
"use server";

export async function myAction(prevState, formData) {
  const data = formData.get("fieldName");
  if (!data) return { success: false, message: "Field required" };
}
```

```
// Logic server
return { success: true, message: "Success" };
}
```

4. Gọi Server Action từ sự kiện khác (non-form)

- Ví dụ với `onClick` :

```
"use client";
import { useState } from "react";
import { logMessage } from "./actions";

function MyComponent() {
  const [response, setResponse] = useState(null);
  const [isLoading, setIsLoading] = useState(false);

  const handleClick = async () => {
    setIsLoading(true);
    try {
      const result = await logMessage("Hello");
      setResponse(result);
    } catch (error) {
      setResponse({ success: false, message: error.message });
    } finally {
      setIsLoading(false);
    }
  };

  return <button onClick={handleClick} disabled={isLoading}>Send</button>;
}
```

- **Server Action:**

```
"use server";

export async function logMessage(message) {
  console.log(`Received: ${message}`);
  await new Promise((resolve) => setTimeout(resolve, 700));
  return { success: true, message: `Received: ${message}` };
}
```

5. Lợi ích

- **Giảm Boilerplate:** Loại bỏ code lặp lại (fetch, state management).
- **Progressive Enhancement:** Form hoạt động ngay cả khi JavaScript bị tắt.
- **Xóa cache linh động:** Dễ dàng làm mới dữ liệu sau khi action hoàn tất.

6. Khi nào sử dụng?

- **Server Actions:** CREATE, UPDATE, DELETE operations.
- **Fetch:** READ operations.

7. Lưu ý

- `"use server";` phải ở đầu file hoặc hàm.
- Server Actions chạy trên server, kiểm tra console server để debug.
- Sử dụng `useActionState` để quản lý trạng thái và phản hồi từ server.

- Xử lý lỗi trong Server Action để trả về thông báo phù hợp.